

Evaluación Parcial N°1

Diseño de Arquitectura de Microservicios

SmartLogix — Plataforma Inteligente para la Gestión Logística de eCommerce

Asignatura: Desarrollo Fullstack III — DSY1106

Integrantes : Catrina Corral

Anais Aravena

Fernanda Manríquez

Sección: 002D

Fecha: Marzo 2026

1.Contextualización del Caso	3
2. Selección de Patrones de Arquitectura.....	3
2.1 Esquema de patrones de arquitectura.....	4
2.2 Justificación de los patrones seleccionados.....	4
3. Herramientas, Estrategias y Comunicación entre Microservicios.....	5
3.1 Justificación de la Selección de Herramientas.....	5
3.2 Esquema de Comunicación entre Microservicios.....	5
3.3 Comunicación entre Microservicios.....	5
4. Seguridad, Privacidad y Sostenibilidad.....	6
4.1 Seguridad.....	6
4.2 Privacidad.....	6
4.3 Sostenibilidad.....	6
5. Esquema de Arquitectura de Contenedores con Docker.....	7
5.1 Arquitectura de Contenedores con Docker.....	7
6. Evaluación General del Diseño Propuesto.....	7
7. Bibliografía.....	8
8. Conclusión.....	9

Contextualización del Caso

SmartLogix surge como una solución tecnológica orientada a optimizar la gestión logística de pequeñas y medianas empresas dentro del ecosistema eCommerce. Actualmente, muchas organizaciones operan bajo arquitecturas monolíticas, lo que genera problemas como falta de escalabilidad, dificultades en la integración con otros sistemas y errores en la sincronización de información.

Estas limitaciones afectan directamente la eficiencia operativa, provocando retrasos en la gestión de pedidos, inconsistencias en el inventario y una mala experiencia para el cliente final.

Frente a este escenario, se propone una arquitectura basada en microservicios, donde el sistema se divide en componentes independientes que permiten una mayor flexibilidad, escalabilidad y mantenibilidad.

Los principales componentes del sistema son:

- **Microservicio de Inventario**, encargado de controlar el stock en tiempo real.
- **Microservicio de Pedidos**, responsable de validar y gestionar órdenes.
- **Microservicio de Envíos**, encargado de la logística de despacho.
- **Microservicio de Usuarios**, encargado de la validación de credenciales, generación de tokens JWT y control de acceso al sistema.
- **Servicio de Notificaciones**, encargado de enviar avisos al usuario sobre el estado de sus pedidos, confirmaciones y actualizaciones relevantes del sistema.

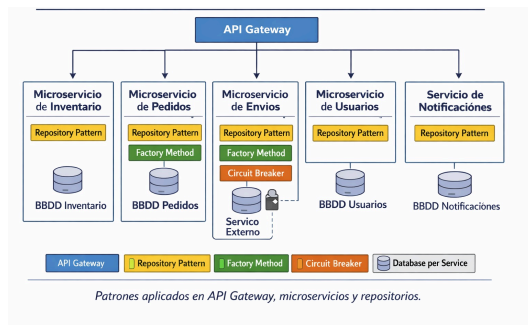
Esta arquitectura permite responder de manera eficiente a los requerimientos del cliente, facilitando la integración con plataformas externas y mejorando el rendimiento del sistema.

2. Selección de Patrones de Arquitectura

En la solución propuesta se utilizan distintos patrones de arquitectura alineados con los requerimientos del sistema SmartLogix. El API Gateway se aplica a todos los módulos, centralizando el acceso y la autenticación mediante JWT. El Repository Pattern se utiliza en los microservicios de Inventario, Pedidos, Envíos, Usuarios y Notificaciones, desacoplando la lógica de negocio de la persistencia de datos. El Factory Method se implementa en el procesamiento de pedidos y la coordinación de envíos, permitiendo gestionar la creación de estos de forma

flexible. El Circuit Breaker se utiliza en el microservicio de Envíos para la comunicación con servicios externos, evitando fallos en cascada. Finalmente, el patrón Database per Service se aplica a todos los microservicios, asegurando que cada uno gestione su propia base de datos, garantizando independencia y escalabilidad.

2.1 Esquema de patrones de arquitectura



2.2 Justificación de los patrones seleccionados

Se selecciona el patrón API Gateway debido a que permite centralizar el acceso al sistema SmartLogix, evitando que los clientes deban conocer la ubicación de cada microservicio, además de facilitar la autenticación mediante JWT y el control de solicitudes (Mostapha, 2025). En SmartLogix, el API Gateway recibe las solicitudes del frontend y las dirige al microservicio correspondiente, como Pedidos o Inventario.

El Repository Pattern permite desacoplar la lógica de negocio de la base de datos en los microservicios de Inventario, Pedidos, Envíos, Usuarios y Notificaciones, facilitando la mantenibilidad del sistema y permitiendo posibles cambios tecnológicos sin afectar la arquitectura (Kranio, 2024). Por ejemplo, el microservicio de Inventario utiliza este patrón para acceder a la base de datos sin exponer directamente la lógica de persistencia.

El Factory Method se utiliza en el procesamiento de pedidos y la coordinación de envíos, permitiendo gestionar distintos tipos de pedidos y envíos sin el uso excesivo de estructuras condicionales, mejorando la extensibilidad del sistema (Gamma et al., 1994). En el microservicio de Pedidos, este patrón permite crear distintos tipos de pedidos como PedidoNormal o PedidoExpress sin modificar la lógica principal.

El Circuit Breaker se aplica en el microservicio de Envíos, específicamente en la comunicación con servicios externos como transportistas, permitiendo detectar fallos y retornar respuestas alternativas, evitando fallos en cascada (AWS, s.f.). Por ejemplo, si el servicio externo de transportistas falla, el sistema responde con un estado alternativo sin interrumpir el flujo del pedido.

Finalmente, el patrón Database per Service permite que cada microservicio gestione su propia base de datos MySQL, asegurando independencia, aislamiento y escalabilidad del sistema (AWS, s.f.). Por ejemplo, cada microservicio accede únicamente a su propia base de datos, sin depender de las bases de datos de otros servicios.

3. Herramientas, Estrategias y Comunicación entre Microservicios

3.1 Justificación de la Selección de Herramientas

La arquitectura propuesta se apoya en herramientas modernas ampliamente utilizadas en la industria.

Se utiliza Spring Boot como framework backend, el cual permite desarrollar microservicios basados en REST, integrando JPA, seguridad y el patrón MVC. Esto permite manejar múltiples solicitudes simultáneas sin degradar el rendimiento del sistema, mejorando la eficiencia en el procesamiento de pedidos.

Docker se emplea para contenerizar cada microservicio junto con sus dependencias, facilitando la portabilidad del sistema entre distintos entornos.

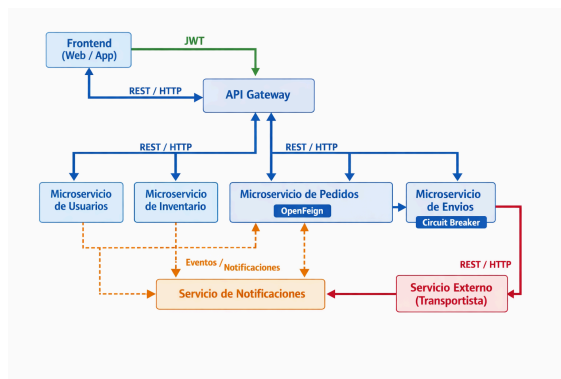
Kubernetes actúa como orquestador, automatizando el despliegue, el escalado horizontal y la recuperación ante fallos. Esto permite la recuperación automática ante fallos, por ejemplo, si el microservicio de Envíos falla, el sistema levanta una nueva instancia sin intervención manual, reduciendo el tiempo de inactividad.

Para la comunicación entre microservicios, se utiliza Spring Cloud OpenFeign, facilitando el consumo de APIs REST mediante interfaces declarativas.

Finalmente, se utiliza MySQL junto con JPA como solución de persistencia, permitiendo que cada microservicio gestione su propio esquema de base de datos bajo el patrón Database per Service. Esto permite realizar consultas eficientes y mantener una estructura organizada de los datos.

Estas herramientas son utilizadas en arquitecturas modernas por empresas como Netflix y Spotify, lo que respalda su eficiencia en entornos reales (Richardson, 2018).

3.2 Esquema de Comunicación entre Microservicios



3.3 Comunicación entre Microservicios

La arquitectura utiliza comunicación síncrona basada en REST/HTTP para operaciones que requieren respuesta inmediata, como validación de usuarios y verificación de pedidos. Para ello, se utiliza Spring Cloud OpenFeign, facilitando la comunicación entre microservicios mediante interfaces declarativas.

El microservicio de Pedidos coordina el flujo principal del sistema, comunicándose con Inventario, Usuarios y Envíos para validar información y gestionar órdenes. Por su parte, el servicio de Notificaciones informa al usuario sobre eventos relevantes, como el estado de los pedidos.

4. Seguridad, Privacidad y Sostenibilidad

4.1 Seguridad

La seguridad se implementa mediante autenticación basada en tokens JWT. Al iniciar sesión, cada usuario recibe un token con sus roles y permisos específicos, el cual es validado por cada microservicio antes de procesar la solicitud.

Se aplica el principio de mínimo privilegio, donde cada usuario solo puede acceder a las funciones correspondientes a su rol (por ejemplo, un operador de bodega solo puede actualizar stock).

Toda la comunicación se protege mediante protocolos seguros HTTPS/TLS, y cada transacción se registra con información como usuario, timestamp e IP de origen, permitiendo la detección de posibles anomalías.

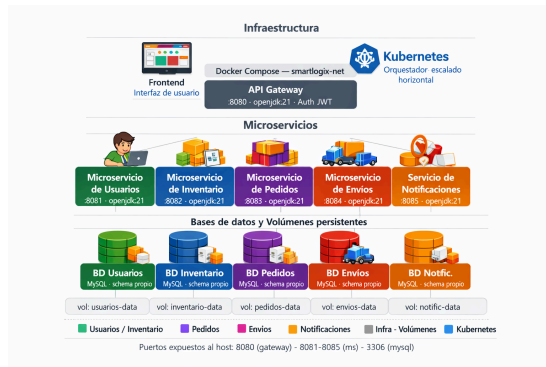
4.2 Privacidad

Los datos de clientes (direcciones, pedidos, contactos) se gestionan de forma segmentada: cada microservicio accede solo a la información que necesita. El área de envíos no puede ver datos financieros. Se cumple con la normativa de privacidad vigente en Chile.

4.3 Sostenibilidad

La arquitectura de microservicios permite que cada componente evolucione de forma independiente, reduciendo la deuda técnica y facilitando la incorporación de nuevas tecnologías sin afectar el sistema completo. Además, el uso de contenedores permite optimizar el consumo de recursos, ejecutando únicamente los servicios necesarios según la demanda. Esto permite reducir el uso energético en períodos de baja carga y mejorar la eficiencia operativa del sistema a largo plazo.

5. Esquema de Arquitectura de Contenedores con Docker



5.1 Arquitectura de Contenedores con Docker

La arquitectura utiliza Docker para contenerizar cada microservicio junto con sus dependencias y su base de datos MySQL, asegurando consistencia entre entornos de desarrollo, pruebas y producción. Cada servicio se empaqueta mediante imágenes, contenedores y Dockerfile, permitiendo ejecutar aplicaciones de forma estandarizada y portable entre distintos entornos (Richardson, 2018). Como se observa en el diagrama, los microservicios de Usuarios, Inventario, Pedidos, Envíos y Notificaciones se ejecutan de forma desacoplada en contenedores independientes, facilitando el mantenimiento y la evolución del sistema. Kubernetes actúa como orquestador, automatizando el despliegue, el escalado horizontal y la recuperación ante fallos, permitiendo escalar de forma selectiva los servicios más demandados, como Pedidos en momentos de alta carga, optimizando el uso de recursos y mejorando el rendimiento general del sistema (Richardson, 2018).

6. Evaluación General del Diseño Propuesto

La arquitectura propuesta responde a los requerimientos del caso mediante microservicios independientes que aseguran modularidad y separación de responsabilidades (Richardson, 2018). La comunicación se realiza mediante REST/HTTP con Spring Cloud OpenFeign, permitiendo una interacción clara y mantenible entre los servicios. En el flujo principal, Usuarios valida la identidad vía JWT, Pedidos gestiona la orden, Inventario verifica el stock, Envíos coordina el despacho y Notificaciones informa al usuario sobre el estado del pedido; ante fallos en servicios externos, el Circuit Breaker evita errores en cascada (AWS, s.f.). Respecto a la sincronización de

inventario en tiempo real, la arquitectura lo resuelve mediante comunicación REST entre Inventario y Pedidos, permitiendo mantener la información actualizada, aunque esta dependencia puede generar latencia si algún servicio no se encuentra disponible.

Como debilidad, la arquitectura presenta mayor complejidad operativa que un monolito, requiriendo equipos con experiencia en sistemas distribuidos y mayor esfuerzo de mantenimiento inicial. Como riesgo, un diseño deficiente de interfaces puede generar acoplamiento entre servicios, y una configuración incorrecta del Circuit Breaker podría afectar la continuidad del servicio (Nygard, 2018). Se recomienda implementar Service Discovery (Eureka) para reducir el acoplamiento, además de incorporar pruebas de contrato y definir políticas de fallback para el Circuit Breaker.

7. Bibliografía

Mecalux. (s/f). La arquitectura de microservicios y sus ventajas en el desarrollo de software. Mecalux.es. Recuperado el 3 de abril de 2026, de <https://www.mecalux.es/blog/microservicios-arquitectura>

Miralpeix, D. (2026, enero 7). Qué es una arquitectura de microservicios. Flexygo; CEESi Asesores. <https://www.flexygo.com/que-es-una-arquitectura-de-microservicios/>

Gil, S. (2020, diciembre 11). Cuando elegir microservicios para construir tu e-commerce. Blog de hiberus; Hiberus. <https://www.hiberus.com/crecemos-contigo/cuando-elegir-microservicios-para-construir-tu-e-commerce/>

de Yavé, V. M. C. (s/f). Patrones de arquitectura en microservicios: configuración... Paradigmadigital.com. Recuperado el 3 de abril de 2026, de <https://www.paradigmadigital.com/dev/patrones-arquitectura-microservicios-configuracion-externa/>

Susnjara, S., & Smalley, I. (2025, noviembre 27). Patrones de diseño para microservicios. Ibm.com. <https://www.ibm.com/mx-es/think/topics/microservices-design-patterns>

P of EAA. (s/f). Martinowler.com. Recuperado el 3 de abril de 2026, de <https://martinfowler.com/books/ea.html>

Building Microservices, 2nd Edition. (s/f). Samnewman.io. Recuperado el 3 de abril de 2026, de https://samnewman.io/books/building_microservices_2nd_edition/

Amazon Web Services. (s.f). Database-per-service patrón. https://docs.aws.amazon.com/es_es/prescriptive-guidance/latest/modernization-data-persistence/database-per-service.html

Mostapha, M. (2025, 10 de abril). Microservicios con Spring Cloud - Arquitectura completa. <https://www.mostapha.dev/blog/microservicios-spring-cloud-arquitectura-completa>

Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). *Design patterns: Elements of reusable object-oriented software*. Addison-Wesley.

Nygard, M. T. (2018). *Release It!: Design and deploy production-ready software*. Pragmatic Bookshelf.

Hohpe, G., & Woolf, B. (2003). *Enterprise integration patterns*. Addison-Wesley.

8. Conclusión

El trabajo realizado en esta primera etapa permitió analizar la problemática de SmartLogix y proponer una arquitectura de microservicios técnicamente fundamentada. La selección de patrones como Repository Pattern, Factory Method, Circuit Breaker y Database per Service responde directamente a los requerimientos del caso, permitiendo construir una solución modular, escalable y desacoplada.

La incorporación de contenedores Docker establece una base sólida para el despliegue y la escalabilidad del sistema, facilitando su crecimiento futuro. Además, las consideraciones de seguridad, privacidad y sostenibilidad integradas desde el diseño inicial permiten asegurar una solución eficiente, confiable y alineada con las necesidades del cliente.

Este informe constituye el punto de partida para las siguientes etapas del proyecto, donde se desarrollarán e integrarán los componentes del sistema, validando en la práctica las decisiones arquitectónicas propuestas.